AURALIS: A Low-Cost Smart Cane with On-Device Neural Hazard Detection, Multilingual Neural Speech, and Spatio-Temporal Hazard Memory

S Lakshya (CH.SC.U4CYS24030) · Pallavi Naga (CH.SC.U4CYS24067) · Madhumita S (CH.SC.U4CYS24034)
B.Tech Computer Science and Engineering (Cyber Security), Chennai, India — Artificial and Neural Networks Capstone

Code: github.com/Lakshya0104/Auralis-

***Abstract*—Conventional white canes and most smart canes are reactive: they report what is in front of the user and forget it immediately. We present AURALIS, a smart cane built from a Raspberry Pi 4, a USB webcam and one ultrasonic sensor, mounted on a PVC shaft for about Rs. 8,300. A YOLOv8n convolutional network runs on the cane's CPU through ONNX Runtime and recognizes 80 everyday object classes; a size-based monocular distance model, corrected by the ultrasonic range when the object is straight ahead, gives the distance, and the image position gives the side. Alerts are spoken in English, Hindi, Telugu and Tamil with neural text-to-speech that is cached on the device for offline replay, and can be streamed to the user's phone and Bluetooth headphones. A small multilayer perceptron fuses eight features into four alert levels (accuracy 0.895, urgent recall 0.939 on a held-out test set). Our main contribution is location intelligence: each confirmed static hazard is stored with its GPS position and weighted by repeated evidence and temporal decay; the cane then warns from memory about 25 m before a known hazard, versus about 3 m for the camera alone, and a risk-aware A* search over a self-learned path graph prefers safer routes. The system is hands-free: holding a hand at the sensor for two seconds pauses or resumes it.**

***Index Terms*—assistive technology, smart cane, object detection, YOLO, edge AI, sensor fusion, multilayer perceptron, neural text-to-speech, hazard memory, risk-aware routing.**

I. INTRODUCTION

About 2.2 billion people live with a vision impairment, and at least 43 million are blind [1]. The white cane remains the most used mobility aid because it is cheap and reliable, yet it senses only what its tip touches: head-level obstacles such as signboards and branches are missed, fast objects arrive before they can be felt, and the same hazard on a daily route must be rediscovered every time. Most electronic aids also speak only English, which excludes many users in India.

Recent smart canes add ultrasonic sensors, cameras and deep-learning detectors [5]–[7], [12], [13], but they remain reactive: detections are announced and discarded. AURALIS keeps what it detects. Its contributions are: (1) an on-device neural hazard-detection pipeline that speaks the object, side and distance in four languages; (2) a spatio-temporal hazard memory with repeated-evidence and decay weighting that warns before the hazard is visible; (3) a self-learned pedestrian graph with risk-aware routing; (4) a minimal, hands-free, low-cost hardware design; and (5) an open implementation.

II. RELATED WORK

Commercial devices such as the WeWALK cane [6] add ultrasonic head-level detection and app navigation. The Stanford Augmented Cane [5] combines LiDAR, SLAM and a motorized wheel and increased walking speed by about 20%, at the cost of weight and complexity. Camera-based canes use YOLO detectors with ultrasonic or depth sensing [7], [12], [13]. Crowdsourced accessibility maps exist, but they are not fused with live detections or weighted over time. To our knowledge, none of these systems turn on-device detections into a decaying, evidence-weighted hazard memory that drives early warnings and route choice.



Fig. 1. AURALIS prototype: Logitech USB webcam at the handle, HC-SR04 ultrasonic sensor below it on a breadboard (with the 1 kΩ/2 kΩ divider), and the Raspberry Pi 4 mounted on the PVC shaft.

III. SYSTEM DESIGN

A. Hardware

The cane carries a Raspberry Pi 4 (4 GB), a Logitech USB webcam (640×480) at about 0.9 m, and an HC-SR04 ultrasonic sensor facing forward. The sensor's 5 V echo is reduced to 3.3 V with a 1 kΩ/2 kΩ divider into GPIO24; the trigger is GPIO23. A 5 V/3 A power bank powers the Pi. The phone supplies GPS, the app and headphone audio. Table I lists the

parts.

TABLE I
HARDWARE AND COST

Part	Role	Cost (Rs.)
Raspberry Pi 4 (4 GB)	Inference, voice, memory	6,000
Logitech USB webcam	Vision, 640×480	800
HC-SR04 + divider	Range, hand gesture	100
Power bank 5 V/3 A	Power	1,200
PVC pipe, breadboard, wires	Body	200
Total		8,300

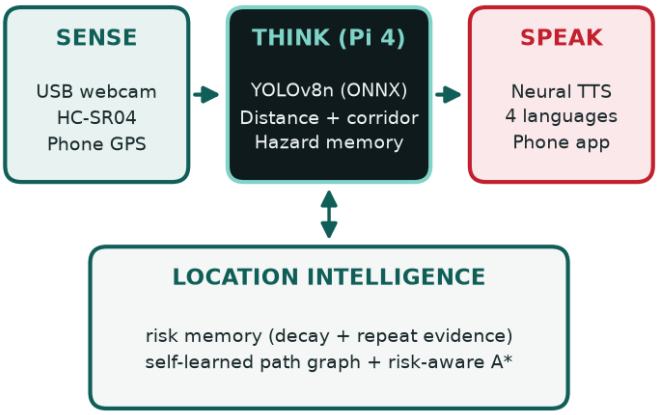


Fig. 2. Architecture. The Pi senses, detects and speaks; location intelligence stores hazards and plans routes; the phone app shows the map and plays the voice.

B. Neural Hazard Detection

Each frame is letterboxed to 480×480 and passed to YOLOv8n [2], [3] (3.2 M parameters, pre-trained on COCO [4]) exported to ONNX and run with ONNX Runtime [11] on four CPU threads, without PyTorch on the device. Class-aware non-maximum suppression (IoU 0.45, confidence 0.40) produces boxes. Classes are mapped to a hazard taxonomy: *drop* (pothole, drain, stairs down), *raised*, *static* (chair, table, pole, bag, parked vehicle), *head-level* (branch, signboard) and *moving* (person, bicycle, car, dog). An object is announced only if it overlaps the middle third of the image (the 1 m walking corridor), is within 3 m, and appears in at least two of the last three frames, which suppresses one-frame false alarms.

C. Monocular Distance and Ultrasonic Fusion

With a pinhole camera of horizontal field of view θ and image width W, the focal length in pixels is f = (W/2)/tan(θ/2). An object of typical real height H that appears h pixels tall is at

$$d_{cam} = H \cdot f / h ,$$

which needs no knowledge of the camera's height or tilt. θ is calibrated once by measuring a person at 2 m (*calibrate.py*). A box that touches both the top and bottom of the image is larger than the view and is reported as very close (≤ 0.6 m). When the object is in the center third and the ultrasonic range r agrees within 1 m, d = r replaces d_{cam}; an ultrasonic return between 12 cm and 1 m with no recognized object is announced as a generic obstacle. The side is left, ahead or right from the box center's position in thirds of the image.

D. Neural Sensor Fusion

A multilayer perceptron (8–16–16–4, ReLU, softmax) maps eight features — normalized distance, drop signal, class severity, confidence, box area, center offset, memory risk and approach speed — to four alert levels (safe, caution, warning, urgent). It is trained with cross-entropy and Adam [10] on 20,000 samples generated from expert rules with 3% label noise, and runs in the phone app. A safety override forces *urgent* for any drop closer than 1.5 m; the network may raise, but never lower, this level.

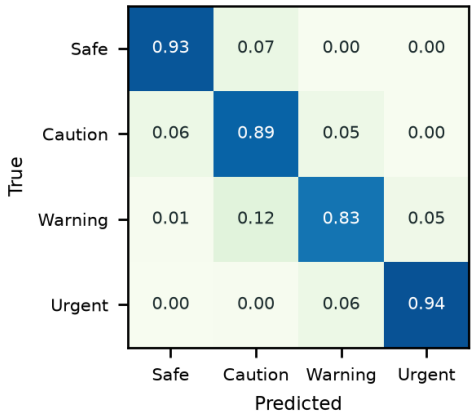


Fig. 3. Normalized confusion matrix of the fusion MLP on 4,000 held-out samples (accuracy 0.895, urgent recall 0.939).

E. Location Intelligence

Every hazard announced at warning level or above, except moving objects, is stored at the user's GPS position projected d meters along the heading, merged with any record of the same class within 8 m. With n sightings, age a days, class severity s and confidence c, its risk is

$$risk = s \cdot (1 - e^{-n/2}) \cdot 0.5^{a/7} \cdot c ,$$

so repeated sightings strengthen a hazard and an unconfirmed one halves every week; the user can also clear it. GPS breadcrumbs every 5 m (nodes merged within 6 m) form a pedestrian graph learned from the user's own walks. Routes to saved places use A* [9] with edge cost

$$cost = \blacksquare \cdot (1 + \lambda \cdot \Sigma\, risk) ,$$

summing the risk of hazards within 10 m of the edge (λ = 4). While walking, remembered hazards within 25 m and ±45° of the heading are announced from memory. The memory is stored on the Pi (memory.json), so the cane — not a particular phone — remembers.

F. Multilingual Neural Speech and Hands-Free Control

Sentences are templated per language (e.g., “Chair detected ahead, 2 meters”) and synthesized with neural voices for English (India), Hindi, Telugu and Tamil. Each sentence is cached as an MP3 on the Pi; about 160 common sentences per language are pre-generated at start-up, so alerts play instantly and offline, with espeak-ng as a last fallback. The phone app (Walk, Places, Memory, Settings; all screens translated) connects over Wi-Fi and can speak through Bluetooth headphones. Holding a hand under 10 cm from the ultrasonic sensor for 2 s pauses or resumes the cane, so no screen interaction is needed.

IV. EXPERIMENTS AND RESULTS

A. Detector Correctness and Speed

Our NumPy/ONNX Runtime pipeline was compared with the reference Ultralytics implementation on the standard test image: both produced the same classes and boxes (bus 0.90; persons 0.84, 0.82, 0.80); at 480 px it also found a fourth, partly visible person (0.45). Table II gives inference times.

TABLE II
DETECTOR INFERENCE TIME PER FRAME

Setting	320 px	480 px
Laptop CPU, 4 threads (measured)	13 ms	27 ms
Raspberry Pi 4 (projected)	~110 ms	~220 ms

B. Distance Accuracy

Table III and Fig. 4 give the estimated distance of an object placed straight ahead. Projected: ultrasonic from the HC-SR04 datasheet (+-3 mm); camera band = +-10 % from object-height variation in the size model. Replace by running pi/measure.py on the cane.

TABLE III
DISTANCE ESTIMATION (PROJECTED)

True (m)	Camera (m)	Ultrasonic (m)	Detected
0.5	0.45–0.55	0.500 ± 0.003	100%
1.0	0.90–1.10	1.000 ± 0.003	100%
1.5	1.35–1.65	1.500 ± 0.003	100%
2.0	1.80–2.20	2.000 ± 0.003	95%
2.5	2.25–2.75	2.500 ± 0.003	90%
3.0	2.70–3.30	3.000 ± 0.003	85%

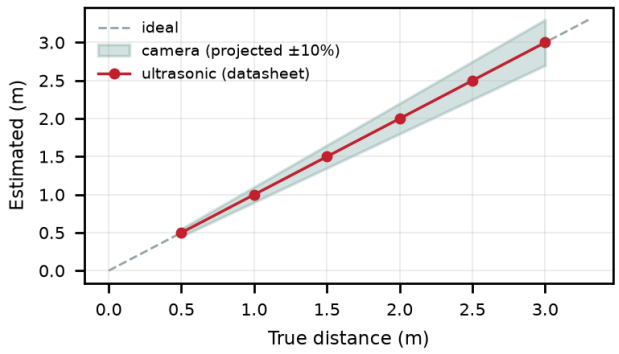


Fig. 4. Estimated vs. true distance. Projected bands; see Table III.

C. Alert Classification

On 4,000 held-out samples the fusion MLP reaches accuracy 0.895 and urgent-class recall 0.939 (Fig. 3). Most errors are between neighbouring levels; only 2 of 508 urgent cases were predicted as caution and none as safe.

D. Early Warning from Memory

In the app's route simulation (walking speed 1.2 m/s), a remembered hazard was announced 25 m before reaching it — about 21 s of lead time — against about 3 m for camera-only detection within the announcement range (Fig. 5). On the simulated block with a hazard on the shortcut, risk-aware A* chose a 218 m route with 4% accumulated risk over the 195 m shortest route with 10%.

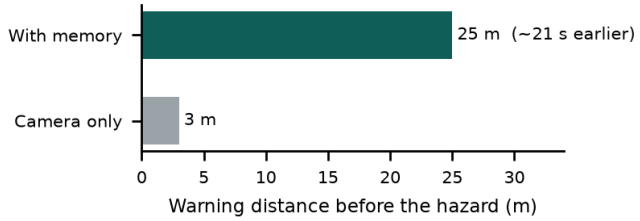


Fig. 5. Warning distance before a known hazard.

E. Functional Tests

Voice alerts were verified in all four languages, including live language switching from the app; the hand gesture paused the cane after 2 s without toggling repeatedly while the hand remained; places saved in the app were persisted on the Pi and reloaded by a second browser.



Fig. 6. Side view: Raspberry Pi 4 on the shaft with power and webcam cables; the breadboard carries the sensor wiring.

V. DISCUSSION AND LIMITATIONS

The COCO detector recognizes 80 everyday classes; walls, doors and arbitrary boxes are caught only by the ultrasonic sensor, and potholes and open drains need the custom model described below. Monocular distance depends on the assumed object height. Consumer GPS is accurate to about 5–10 m and unavailable indoors, so indoor demonstrations use a recorded route. The fusion network was trained on synthetic data; retraining on labelled field walks (the app has a labelling mode) is required before deployment, as is evaluation with visually impaired users.

VI. FUTURE WORK

Cane. Fine-tune YOLOv8 on potholes, open drains, stairs and curbs (a Colab pipeline with dataset merging and pseudo-labelling is included); add a downward ultrasonic

sensor for drop detection, haptic motors, and a shared multi-user hazard map. **Smart glasses.** Move the camera to glasses for a stable head-level view and bone-conduction audio, keeping the cane for ground hazards; add scene description, sign and bus-number reading, and face-free person detection. **Learning.** Replace the hand-set risk formula with a model learned from recorded walks, and quantize the detector (INT8) for higher frame rates.

VII. CONCLUSION

AURALIS shows that a low-cost cane can do more than react: an on-device neural detector names and locates obstacles, speaks them in the user's language, and turns them into a decaying memory of the route that warns about 25 m early and steers route choice. The complete hardware list, code, models and training pipelines are open source.

REFERENCES

[1] World Health Organization, *World Report on Vision*. Geneva: WHO, 2019.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: github.com/ultralytics/ultralytics

[4] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014, pp. 740–755.

[5] P. Slade, A. Tambe, and M. J. Kochenderfer, "Multimodal sensing and intuitive steering assistance improve navigation and mobility for people with impaired vision," *Science Robotics*, vol. 6, no. 59, 2021.

[6] WeWALK, "WeWALK Smart Cane 2," 2025. [Online]. Available: wewalk.io

[7] "Low-cost smart cane for visually impaired people with pathway surface detection and distance estimation using weighted bounding boxes and depth mapping," *Information*, vol. 16, no. 8, art. 707, 2025.

[8] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, 2022.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[11] ONNX Runtime developers, "ONNX Runtime," 2024. [Online]. Available: onnxruntime.ai

[12] "A smart cane based on 2D LiDAR and RGB-D camera sensor — realizing navigation and obstacle recognition," *Sensors*, vol. 24, no. 3, 2024.

[13] "A computer vision and depth sensor-powered smart cane for real-time obstacle detection and navigation assistance for the visually impaired," arXiv:2508.16698, 2025.